MULTI-AGENT AI FOR GAME DEVELOPMENT · ASSIGNMENT #1

Emberkeep

Game Design Document — First Draft

Aaron Low · 21 July 2026 · Status: Draft

1 Executive Summary

Emberkeep is a 1–4 player co-op roguelike dungeon crawler set in the Undervault, a kingdom whose lamp-halls went dark floor by floor a thousand years ago when something at the bottom woke. Each run, a hero descends to rally a growing retinue: every class recruits, awakens, bonds, or kindles allies that fight alongside them, escalating from a handful of units to screen-filling hordes rendered in a 2-bit palette built specifically to stay legible at that scale.

This is a roguelike first. The run is the primary loop, and **retinue growth is the progression axis and the run's reward.** The RTS command layer expresses that power moment-to-moment; the persistent world adds seasoning between runs — both serve the spine.

Loss is hero death — the run ends immediately. **Per-run win** is clearing the floor's boss and reaching the exit. **Meta-win** is longer-term: power resets every run, but the decisions you make persist — they write to a small set of world flags (faction standings, named NPCs' fates, site states) that the next run's procedural

generation reads, so across many runs the Hollow Crown's grip on the Undervault visibly weakens. Those decisions are protected by a firm rule: every world-scarring choice goes to a **party vote**, and a passed vote writes to every present player's own save alike.

The project is built by a small multi-agent AI development pipeline (Section 3): specialized director agents own the lore, the readability rules, and the numbers; MCP tool servers let those agents edit the live Unreal Engine 5.8 project and generate pixel art directly; and a **review board** of six agents adversarially audits the design canon itself — the reason every world-scarring choice requires the whole party's agreement is that this board exists to find exactly that kind of gap. The pipeline has already shipped the proof: the full GDD/CLASSES/WORLD design canon, a scoped RTS vertical slice definition, a completed design-review cycle, and is mid-way through Spike 1, the entity-count technical risk the whole game depends on.

2 Game Mechanics

PLAYER-FACING ACTIONS AND LOOP

1. **Pick your class.** Four working roster slots, each a genuinely different game: **Vanguard** (melee anchor; retinue of liberated soldiers who hold ranks and form shield walls), **Relickeeper** (fortifier; awakens slow, durable stone guardians and inscribes ward zones), **Pathfinder** (ranged skirmisher; a small named hunting pack of bonded beasts and scouts), **Lampbearer** (healer/vision; kindles a drifting constellation of captured souls that heal instead of fight). Every class starts locked to its own retinue identity.
2. **Descend and grow your army.** Enter a procedurally generated floor and fight through combat encounters. Retinue size starts at 3–10 units and can reach the hundreds by late run — grown by rescuing prisoners, awakening dormant guardians, bonding wounded creatures, or kindling lost souls, depending on class. Growth sites scale with party size — a floor guarantees at least one per player, keeping a full 4-player party fed and a solo player's

pace comfortable. Feed your army and it stays strong: every unit draws on a shared supply, and a unit that outruns it visibly *degrades* — dimmer, weaker — until you catch back up. The army is governed by upkeep — a living economy that breathes with the fight.

3. **Command with broad stances.** One button/wheel issues a single order to the whole retinue: **Follow** (default formation), **Charge**, **Hold**, **Rally**. Each class reflavors these same four verbs — the Vanguard's Hold is a literal Shield Wall that blocks enemy pathing; the Pathfinder's Charge, *Loose the Pack*, sends the pack hunting off-screen. The **leash rule** keeps the army with you: any unit that strays too far from the hero — including one on Hold — snaps back to Follow, so anchoring a chokepoint always means staying in the fight beside your troops.

4. **Face a personal decision.** Each floor includes at least one decision event targeted at a specific player: a fork (free the caged prisoners, raising the alarm, vs. loot the vault quietly), a sacrifice (trade retinue lives or hero HP for a power spike), or a moral choice with consequences that follow you past this run. A Vanguard's permadeath Veterans and a Pathfinder's pack member who's merely "found again, changed" carry very different weight — so sacrifice offers price what you'd actually feel losing, giving every class's choice equal weight. Hoarding carries its own cost: a Lampbearer who keeps gathered souls burning watches them dim and grow restless — **unrest**, the mounting toll of temptation.

5. **Clear the floor — the per-run win condition.** Beat the floor's boss and reach the exit. Three floors escalate in density toward a final confrontation.

6. **Die, and the run ends — the loss condition.** Hero death ends the run immediately and finally.

7. **The world remembers — the meta-win condition.** When the run ends (win or death), all hero and retinue power resets to zero for the next run — but the world flags your decisions wrote persist: a site you saved might become a safe room next time; an NPC you spared might reappear as a recruitable ally. Anything that would scar the world for the whole party

goes to a **party vote** (simple majority; ties and absent players default to the status quo), and a passed vote writes to **every present player's own world** alike — every world-altering choice reflects the whole party's will.

3 AI Architecture

AGENTS AND THEIR GAMEPLAY EFFECT

The game is built by a small team of specialized development agents plus two MCP tool servers that give those agents direct hands on the engine and the art pipeline. Each canon file (lore, art specs, systems data) has exactly one agent with write access, and every other agent treats it as read-only — canon conflicts are caught by that structure itself. A seventh role, the review board, audits the canon.

AGENT / TOOL	DEVELOPMENT ROLE	WHAT THE PLAYER EXPERIENCES
`narrative-director`	Writes the world's lore, faction fiction, decision-event text, and NPC dialogue a player reads at event sites, then hands off an art brief for anything visual.	The prisoner you free at a rescue site has a name, a reason to fear the Legion, and a line that changes depending on whether you saved or ignored her last run.
`pixel-art-director`	Defines the 2-bit silhouette and value rules per faction and reviews sprite-sheet layouts for readability, producing palette tables and written specs.	In a 200-unit battle you can tell your Liberated ranks from Legion soldiers from the Quiet's snuffers at a glance, by shape and value alone, before you'd ever need to read a health bar.
`gameplay-director`	Owns the stat blocks, scaling curve, and encounter budget in `SYSTEMS.md` / `docs/data` — the numbers behind "a handful of peasants" becoming a screen-filling army.	Floor 1 feels survivable with three militiamen; floor 3's boss room throws a wave your army can actually break, because someone tuned the curve, not just the individual fight.

AGENT / TOOL	DEVELOPMENT ROLE	WHAT THE PLAYER EXPERIENCES
`unreal-mcp` (tool)	Exposes the live UE 5.8 editor — actors, Blueprints, Niagara systems — over HTTP so an agent can build and wire gameplay directly in-editor instead of just describing it in a document.	The stance wheel, the leash-break-to-Follow behavior, and the swarms you fight in the Spike 1 prototype were all built through this channel — it's how the design canon becomes a playable floor.
`pixellab` (tool)	An MCP-driven generation service that produces character sprites, portraits, tiles, and animations to the pixel-art-director's spec, with every result reviewed before it ships.	The Vanguard's portrait and the Liberated militia sprite are generated here, kept in a holding folder, and only promoted to the game's actual art once a human confirms they read correctly at scale.
Orchestrator	Reads the design canon, briefs each director with only the section it owns, and integrates their output back into the docs and the live UE project.	Keeps three independent agents from drifting out of sync with each other — e.g., a stance-rule change gets propagated everywhere it's referenced instead of quietly rotting in one file.
GDD Review Board (`gdd-review-kit`)	Six isolated-context reviewer agents (systems-designer, narrative-critic, player-psychologist, feasibility-lead, adversarial-qa, business-analyst) independently critique the design canon, cross-examine each other's findings in a second round, then a moderator synthesizes a ranked, ratifiable edit plan.	The reason every shared NPC's fate and every faction's standing reflects the whole party's agreement is that an adversarial-QA pass on this exact document found the gap — "the party owns the scar" is a rule in the game because an agent argued for it.

4 Technical Strategy

ROLES, TOKEN BUDGET, CONSTRAINTS

Model-tier assignment — grounded in measured usage to date. A local usage audit across all development sessions on this project (`scripts/token-usage-report.js`, 3,731 assistant turns, 11 days of active development) shows the actual tier split: **Claude Fable 5** has carried the bulk of turns to date (2,137 turns, ~14.3M fresh tokens) — the high-volume model for open-ended lore, spec-writing, and general development work; **Claude Sonnet 5** is the current default and handles the majority of structured/technical passes (1,531 turns, ~7.2M fresh tokens); **Claude Opus 4.8** is reserved for the highest-stakes passes — cross-file canon integration, and the entire GDD review board, which runs entirely on Opus 4.8 (104 turns, measured from `gdd-review-kit`'s own session logs): adversarial critique of your own design is exactly the high-stakes, low-volume case that tier exists for. **Claude Haiku 4.5** is a plausible future offload for narrow, repetitive calls (tagging a generated asset, updating one stat row).

Token budget — measured, not estimated, via `scripts/token-usage-report.js` for the main project and `gdd-review-kit`'s own session logs for the review board. "Fresh" tokens (input + output + cache-creation) are the meaningful figure — the tokens that represent actual new work. Cache-read tokens (context replayed within a session) are excluded from the projection below: they're billed at a fraction of the fresh-token rate and scale with session length, not with work done.

METRIC	MEASURED VALUE	NOTES
Fresh tokens to date (main dev project)	~22.25M	across 3,731 assistant turns, 2026-07-09–07-19; input 305k + output 4.14M + cache-creation 17.8M
Review board pass	~850k fresh	104 turns, one full five-round cycle, entirely Opus 4.8 — measured from `gdd-review-kit`'s own session logs, a separate project
Cache-read tokens to date	~552.65M (main) + ~6.36M (review)	replayed context, heavily discounted

METRIC	MEASURED VALUE	NOTES
Semester total (projected)	~55–95M fresh tokens + ~850k per review cycle	excluding the Jul-10 outlier, the other 6 logged days averaged ~1.86M fresh/day, extrapolated across 30–50 active-development days over a ~14-week semester; a review pass adds a known, cheap, recurring ~1% on top each time it runs

Named constraints.

- **Single shared UE 5.8 editor instance (the binding constraint).** All in-engine work goes through one live editor process running the `unreal-mcp` server on port 8000. UE-touching work is serialized — one agent, one call, at a time — while directors continue writing docs/specs in parallel. The server also runs in tool-search mode (`list_toolsets` → `describe_toolset` → `call_tool`), a 3-hop indirection per action that costs more tokens than a direct tool call would, in exchange for keeping the exposed tool surface small enough for the model to reason about reliably.
- **Entity-count spike gate — the semester's real go/no-go.** The core hook — hundreds-to-thousands of Mass Entity units at 60fps under 4-player replication — is a hard, enforced gate: content and class production begin once Spike 2 passes at the **full 4-client target load**. A gate failure invalidates the concept, and the evaluated fallbacks are reduced entity counts, single-player-only mass, or a fake-crowd hybrid.
- **Pre-resourcing posture.** Team size, budget, and ship window are explicitly TBD. The v1 scope tables describe intent; costing begins once a team exists.
- **PixelLab is a separate, credit-metered API, distinct from Claude's token budget.** Image generation draws on a PixelLab subscription/credit balance of its own, and every generation — kept or rejected — is saved to `RawArt/Renders/` first, so a later keep/reject decision always has the original to compare against.

Semester-realistic scope. Leaning on `GDD.md` §10's milestone order (Spike 1 → Spike 2 → Spike 3 → RTS vertical slice) and `RTS-VERTICAL-SLICE.md`'s explicit cut lines: **shipped/scoped for this semester** is 1 class (Vanguard), 1 biome (the Highgates), 3 floors, 1 boss, 2 decision events, 2–4 player replication, and a stubbed world-flag write, built against a governed retinue economy and a party-vote multiplayer model. **Explicitly cut for now:** the other 3 classes, the full 15-flag persistent world system, the real loot/evolution system, a pacing-director AI, and — the deliberate scope call this draft makes — the Gatecamp hub as a navigable open-world space between runs. This semester is the **dungeon-crawler half**: procedurally generated floors, combat, decision events, boss, exit. The hub stays a menu until the floor-based loop is proven; a traversable open-world hub becomes the next biome to build once this slice lands.
